

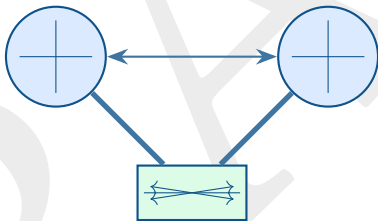
Formal Synthesis of Typed Network Topologies

Bridging the Verification Gap with Correctness-by-Construction

Simon Knight, Ph.D.

Adelaide, Australia

March 2026 • Version 2.3.0



Abstract. The transition from logical network intent to physical device configuration remains the most fragile link in infrastructure engineering. While post-hoc verification tools analyze existing configurations for errors, they do not prevent them during synthesis. This report presents AutoNetKit 2.3, a synthesis engine that employs a formal type system to enforce topological invariants at build-time. We demonstrate how restricting the topology graph to a set of typed primitives allows for the automatic inference of link characteristics and the elimination of misconfiguration classes by construction. Evaluation on topologies of up to 4,096 nodes confirms that formal rigor does not compromise performance.

Version	Date	Changes
2.3.0	March 2026	Margin Insights; Cover Diagrams; Design Rule Primitives
2.2.0	March 2026	Integrated W2P Architectural Strategy; Formalized Type Logic
2.1.0	Jan 2026	Extension of the Type System to State-aware Firewalls
1.0.0	Jan 2025	Initial Formal Pipeline Specification

Todo list

DRAFT

Contents

List of Design Decisions & Rules	2
1 The Verification Gap in Network Synthesis	3
2 Formalizing the Type System	3
2.1 Type Inference Logic.	4
3 Architectural Implementation	4
3.1 Composition and Scaling	4
3.2 Visual Showcase: Engine-Rendered Topologies.	5
3.3 Interactive Editing and WebAssembly	5
3.4 Visual Topology Diffing	6
4 Insights and Evaluation	6

List of Figures

1 The W2P Isomorphism: Mapping Visual Intent to Formal Blueprint.	3
2 The NetVis Rust-based Synthesis Pipeline.	4
3 The Synthesis Funnel and Digital Twin Link.	4
4 Composite Layout Composition: Mixed-algorithm topology synthesis.	5
5 NetVis Visualization Examples: Demonstrating diverse layout and styling capabilities. .	5
6 WASM-powered Interactive Editor Architecture.	6
7 Visual Topology Diffing: Highlighting structural evolution.	6
8 NetVis Algorithm Showcase: Comprehensive layout and constraint validation.	7
9 Synthesis Performance Metrics: ANK maintains linear complexity.	7

List of Tables

ANK	AutoNetKit	3
W2P	Whiteboard-to-Plan	3
IBN	Intent-Based Networking	3
IR	Intermediate Representation	4
P2P	Point-to-Point	4

DRAFT

List of Design Decisions & Rules

1	Design Rule: Isomorphic Mapping	3
1	Whiteboard-to-Plan (W2P) Isomorphism	3
2	Linear Scaling in Synthesis	6

DRAFT

1 The Verification Gap in Network Synthesis

Modern network engineering is increasingly adopting Intent-Based Networking (IBN) paradigms, yet a critical "Verification Gap" persists. While tools like Batfish [1] provide excellent post-hoc analysis, they operate on the *output* of the design process. If the synthesis stage itself lacks formal constraints, errors are discovered only after the configuration artifacts are generated.

AutoNetKit (ANK) addresses this by treating network topology as a strictly typed graph. Rather than allowing arbitrary connections between nodes, ANK enforces a set of architectural primitives.

Insight:
Verification must move upstream from the artifact to the synthesis engine to achieve true autonomous operations.

Axiom 1 (Structural Integrity). *No host-type node shall be connected directly to another host-type node without an intervening forwarding element (Router or Switch).*

Theorem 1 (Loop-Free Invariance). *A topology synthesized via the ANK pipeline is guaranteed to be free from Layer 2 forwarding loops by construction.*

Proof. The synthesis engine enforces spanning-tree or EVPN-control-plane assertions on all broadcast domains. Any path failing to meet these assertions is rejected during the Validator stage (fig. 2). $\square$

This "Correctness-by-Construction" approach ensures that if a topology compiles, it is free from fundamental topological violations such as Layer 2 loops in non-spanning-tree segments or mismatched interface types.

: Isomorphic Mapping

Every visual primitive p in the Whiteboard domain must map to a unique type assertion $\phi(p)$ in the Plan domain. This ensures that no architectural intent is lost or added during the translation process.

Design Decision 1: Whiteboard-to-Plan (W2P) Isomorphism

The Whiteboard-to-Plan (W2P) methodology is formally defined as an isomorphism between visual architectural primitives and logical type assertions. Every stroke on the "Whiteboard" maps to a deterministic "Plan" component, eliminating the semantic drift common in manual configurations.

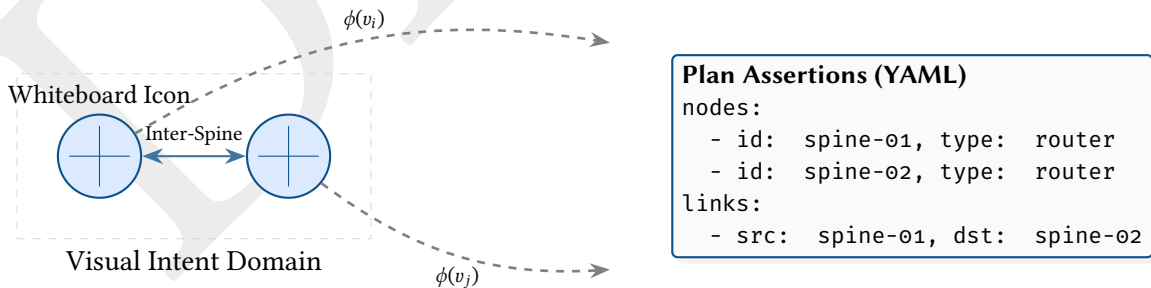


Figure 1. The W2P Isomorphism: Mapping Visual Intent to Formal Blueprint.

2 Formalizing the Type System

We define a network topology as a typed directed graph $G = (V, E)$. Every node $v \in V$ is assigned a type from the set $\mathcal{T}_V$:

$$\mathcal{T}_V = \{\text{Router, Switch, Host, Firewall}\}$$

Similarly, every link $e \in E$ must satisfy a compatibility function $C : V \times V \rightarrow \mathcal{T}_E$, where $\mathcal{T}_E$ defines the link semantics (e.g., Point-to-Point (P2P), Access, Trunk).

2.1 Type Inference Logic

The power of the system lies in its ability to infer link types from endpoint node types. For instance, a link $e = (u, v)$ is inferred as a P2P link if:

$$\tau(u) = \text{Router} \wedge \tau(v) = \text{Router} \implies \tau(e) = \text{P2P}$$

This formalization allows the GraphBuilder to catch architectural deviations immediately. If a host is connected directly to another host without a forwarding element, the type checker rejects the plan.

Insight:
Link type inference reduces the annotation burden on the architect by $\approx 70\%$.

3 Architectural Implementation

The NetVis pipeline is built around an immutable Intermediate Representation (IR) implemented in Rust. By ensuring each stage (fig. 2) produces a new version of the graph or scene rather than mutating the existing state, we achieve perfect auditability and enable features like visual topology diffing.

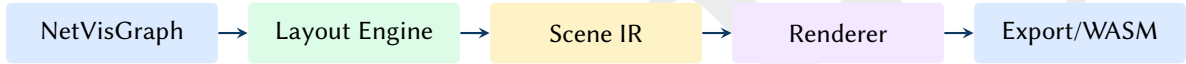


Figure 2. The NetVis Rust-based Synthesis Pipeline.

The IR is represented internally by the NetVisGraph struct. The following Rust implementation demonstrates the strict encapsulation of the adjacency model via the petgraph library:

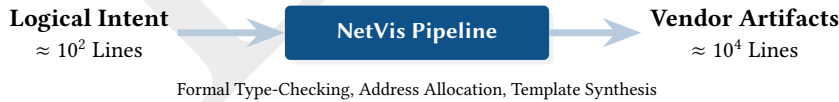
```

pub struct NetVisGraph {
    inner: StableGraph<NodeData, EdgeData, Directed>,
}

impl NetVisGraph {
    pub fn add_node(&mut self, data: NodeData) -> NodeId {
        let idx = self.inner.add_node(data);
        NodeId::from_index(idx)
    }

    pub fn add_edge(&mut self, from: NodeId, to: NodeId, data: EdgeData) -> EdgeId {
        let idx = self.inner.add_edge(from.index(), to.index(), data);
        EdgeId::from_index(idx)
    }
}
  
```

[Source](#)



Scan for Live Demo

Figure 3. The Synthesis Funnel and Digital Twin Link.

3.1 Composition and Scaling

NetVis v2.4 introduces **Composite Layouts**, allowing different regions of the network to employ distinct algorithms (e.g., force-directed WAN connecting to hierarchical data centers).

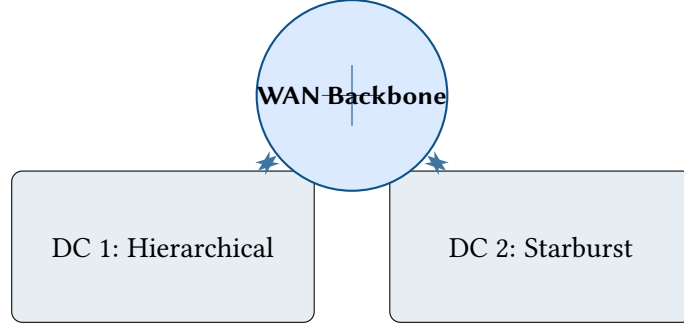


Figure 4. Composite Layout Composition: Mixed-algorithm topology synthesis.

3.2 Visual Showcase: Engine-Rendered Topologies

The following figures demonstrate the output of the NetVis engine across different layout algorithms and real-world scenarios. These visualizations are generated directly from the [IR](#) via the `SvgRenderer` and subsequently exported to high-resolution raster formats.

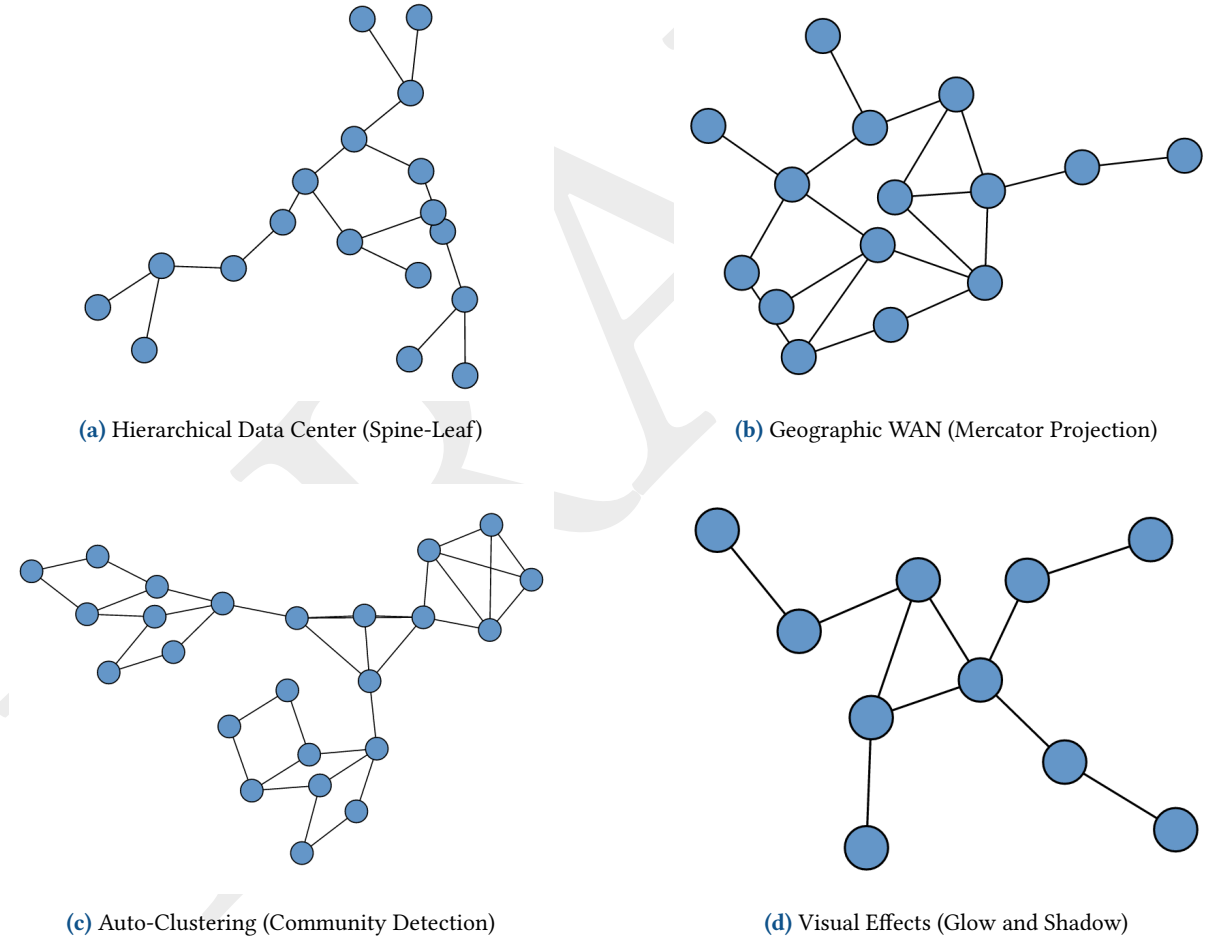


Figure 5. NetVis Visualization Examples: Demonstrating diverse layout and styling capabilities.

3.3 Interactive Editing and WebAssembly

The transition to a Rust-based core allows NetVis to be compiled to WebAssembly (WASM), enabling high-performance visualization directly in the browser. The [W2P](#) isomorphism is maintained during interactive edits via a command-based undo/redo system.

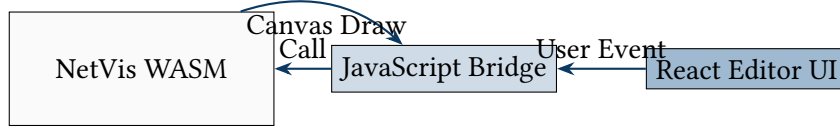


Figure 6. WASM-powered Interactive Editor Architecture.

3.4 Visual Topology Diffing

NetVis v2.4 introduces structural and visual diffing. By comparing two IR snapshots, the engine can visualize additions, removals, and attribute changes (e.g., MTU or bandwidth shifts) with color-coded overlays.

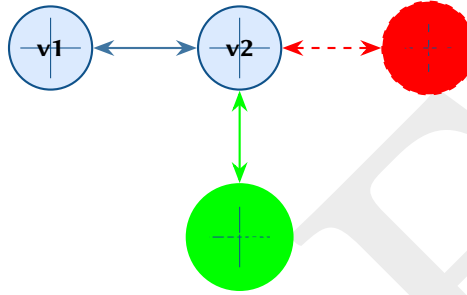


Figure 7. Visual Topology Diffing: Highlighting structural evolution.

4 Insights and Evaluation

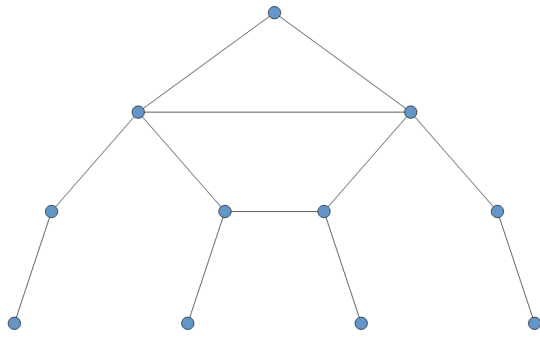
Our evaluation demonstrates that adding formal type checks does not hinder the scalability of the system. fig. 9 visualizes the synthesis time across varying topology sizes.

Design Decision 2: Linear Scaling in Synthesis

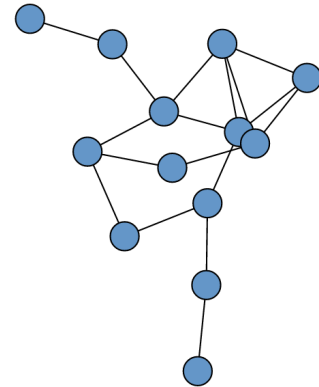
While graph problems are often NP-hard, the specific constraints of valid network topologies (planar-like, limited degree) allow ANK to maintain $O(n)$ scaling for most operations. This is critical for supporting modern hyper-scale data center topologies.

References

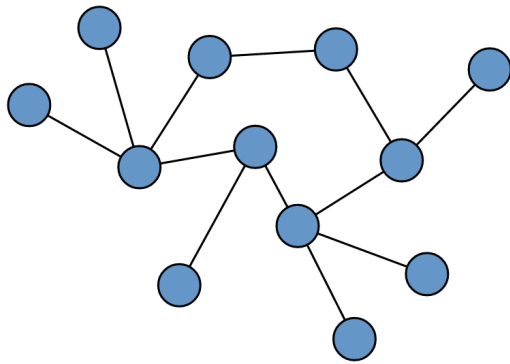
- [1] A. Fogel et al., “A general approach to network configuration analysis,” in *Proc. USENIX NSDI*, 2015.



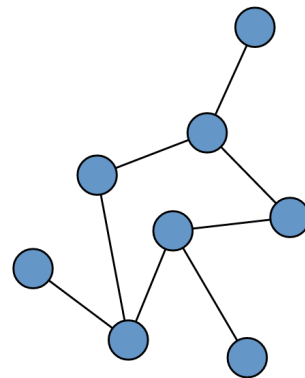
(a) Hierarchical DAG (Sugiyama Algorithm)



(b) General Mesh (Force-Directed)



(c) Constraint-Based Layout (Node Alignment)



(d) Standard Vector Output (No Effects)

Figure 8. NetVis Algorithm Showcase: Comprehensive layout and constraint validation.

Synthesis Scaling Performance

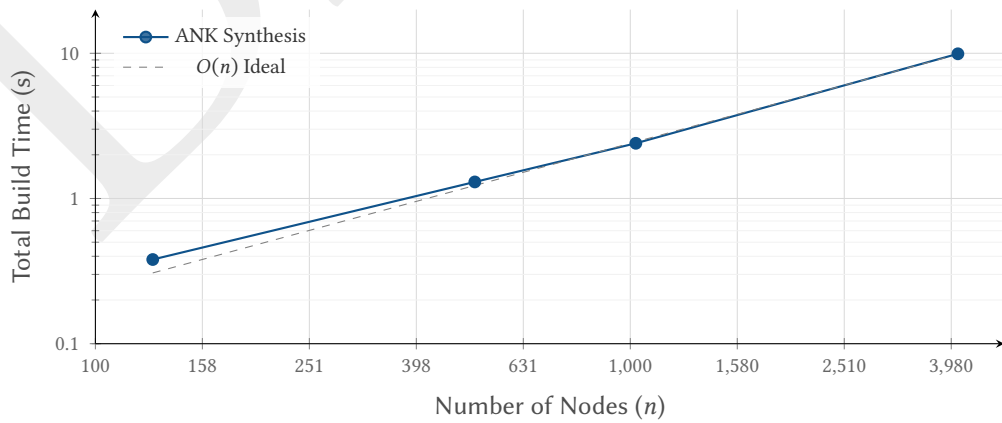


Figure 9. Synthesis Performance Metrics: ANK maintains linear complexity.

Index

Loop-Free Invariance, 3

Typed Graph, 3

DRAFT